

DocRate - 의료 리뷰 및 병원 정보 플랫폼

공공 의료 데이터를 기반으로 병원 및 의사 정보를 제공하고,
사용자가 리뷰를 작성 및 조회할 수 있는 의료 리뷰 플랫폼을 구축하는 프로젝트

기술 스택

Backend

- Java 17
- Spring Boot
- Spring Security
- Spring Data JPA
- JWT
- Thymeleaf

Frontend

- HTML5
- CSS3
- JavaScript

Database / Cache

- MySQL 8
- Redis

Data Processing

- Python
- Pandas

Infra

- Docker
- Docker Compose

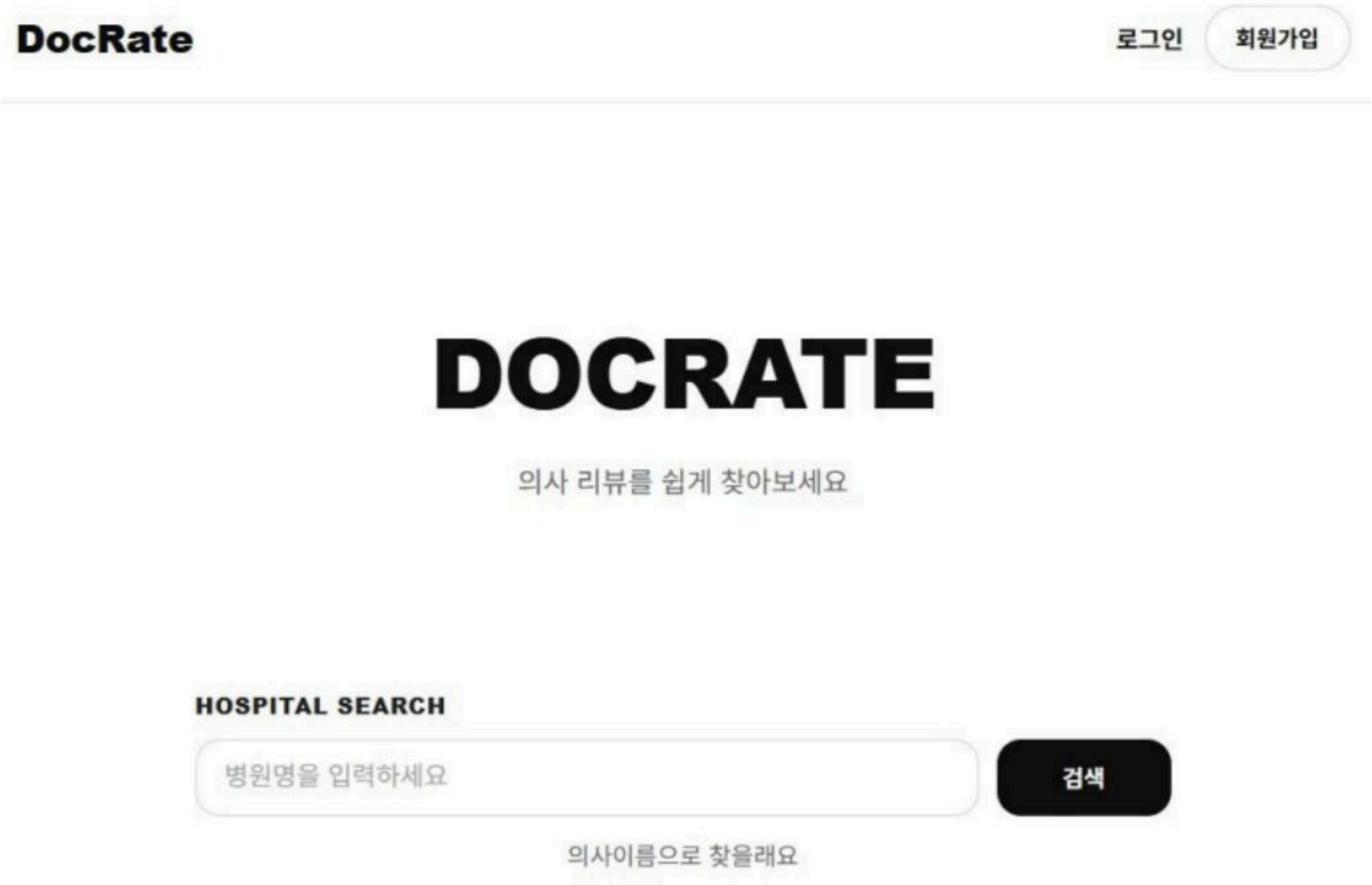
Collaboration

- Git
- GitHub

프로젝트 개요

- **프로젝트명**
DocRate - 의료 리뷰 및 병원 정보 플랫폼
- **프로젝트 소개**
공공 의료 데이터를 기반으로 병원 및 의사 정보를 제공하고, 사용자가 리뷰를 작성·조회할 수 있는 의료 리뷰 플랫폼 개발 프로젝트입니다.
- 단순 CRUD 구현을 넘어 대용량 공공 데이터 전처리 및 적재 자동화, JWT 기반 인증 시스템, Docker 기반 개발 환경 구성, Redis 활용, AWS 기반 확장 가능한 구조 설계까지 실제 서비스 운영 환경을 고려하여 개발하였습니다.
- **개발 인원**
6명
- **담당 역할**
 - **데이터베이스 / 데이터 파이프라인 메인롤**
 - i. 공공 의료 데이터 ETL 파이프라인 설계 (Pandas 기반)
 - ii. 병원·의사·리뷰 도메인 ERD 및 관계형 테이블 설계
 - iii. MySQL 기반 DDL / DML 작성 및 데이터 모델링
 - iv. 데이터 적재 및 테스트용 샘플 데이터 구축
 - v. 데이터 정합성 검증 및 API 테스트 지원
 - **Infra / Docker**
 - i. 컨테이너 기반 데이터 적재·서비스 환경 오케스트레이션
 - **Backend**
 - i. 메인 페이지 기능 개발
 - ii. 병원 조회 API 및 JWT 인증 로직 구현 및 버그 수정

Main 페이지 백엔드 및 프론트엔드 구현:



- **요구사항:**
사용자가 병원 검색 기능과 서비스 주요 기능을 메인 화면에서 직관적으로 사용할 수 있어야 함.
- **작업:**
Spring Boot와 Thymeleaf를 활용하여 Main 페이지의 백엔드 API와 프론트엔드 UI를 구현

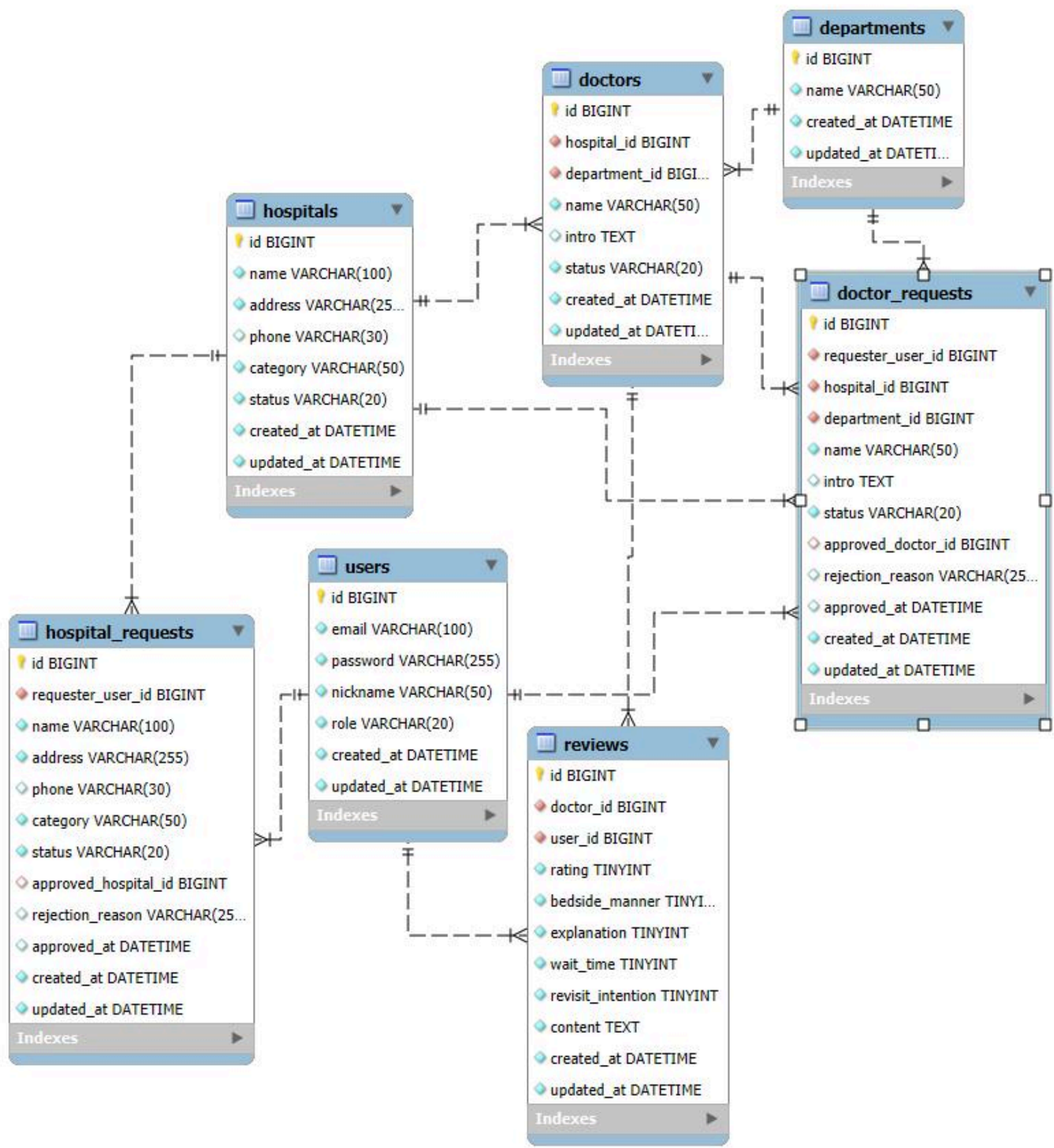
공공 의료 데이터 전처리 및 적재 자동화:

- **요구사항:**
공공 의료 CSV 데이터를 안정적으로 서비스 DB에 저장할 수 있어야 함.
- **작업:**
Pandas 기반 CSV 데이터 전처리 및 MySQL 자동 적재 파이프라인 구축
- **문제:**
데이터 컬럼 구조 및 인코딩이 일정하지 않았으며, 동일 병원 데이터가 중복 저장되는 문제 발생
- **문제 해결:**
 - a. 주소 및 전화번호 데이터 정규화 수행
 - b. 병원명 + 주소 기반 중복 제거 로직 구현
 - c. utf8mb4 charset 적용을 통해 한글 깨짐 문제 해결
 - d. Docker 기반 Data Loader 환경 구성으로 자동 적재 프로세스 구축

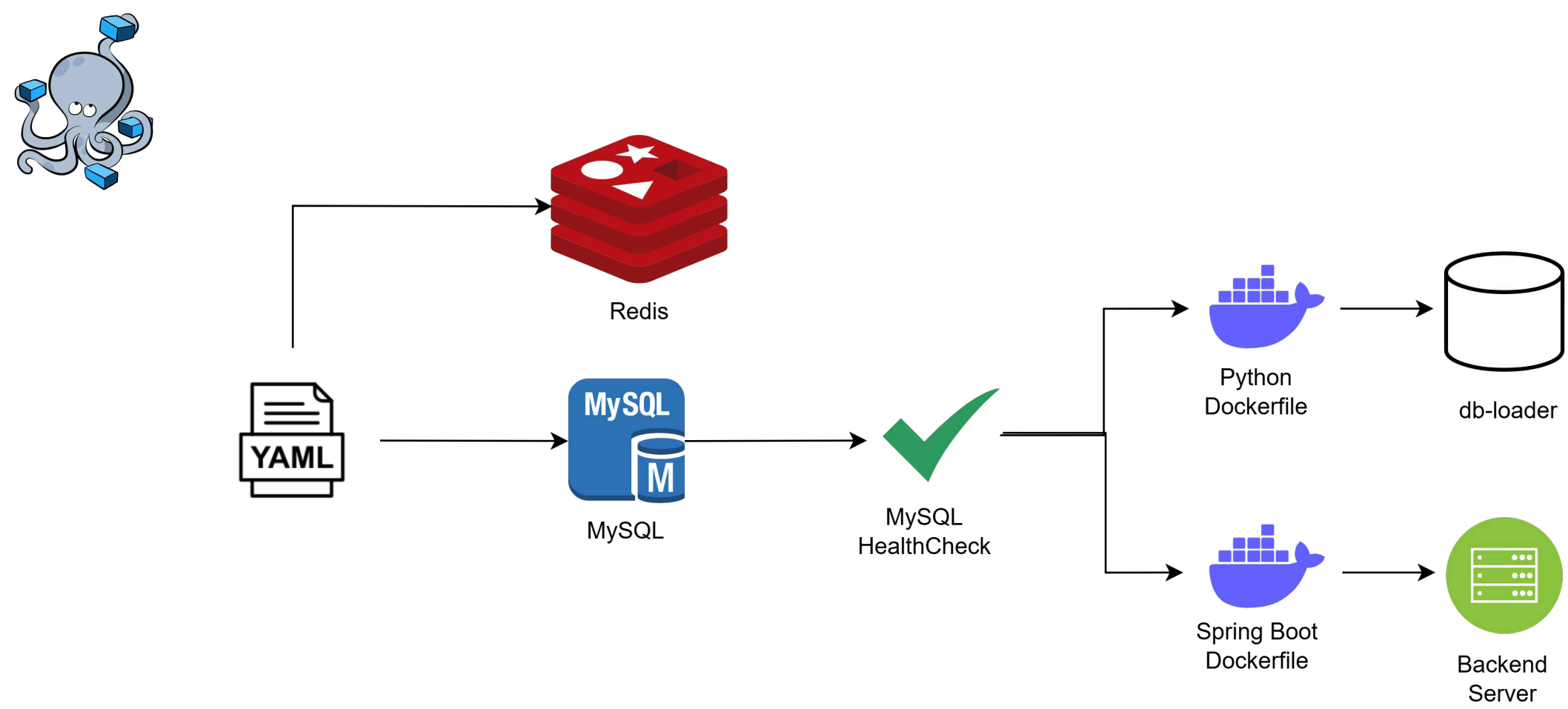
사용자 인증 시스템 구현:

- **요구사항:**
로그인 상태를 유지하면서도 확장 가능한 인증 구조가 필요
- **작업:**
JWT Access Token / Refresh Token 기반 인증 시스템 구현
- **문제:**
Refresh Token이 유효한 상태에서도 Access Token이 정상적으로 재발급되지 않는 문제 발생
- **문제 해결:**
 - a. JWT 인증 Filter 흐름 분석 및 Refresh Token 검증 로직 개선
 - b. Security Context 처리 순서 문제 수정

병원·의사·리뷰 도메인 ERD 및 관계형 테이블 설계:

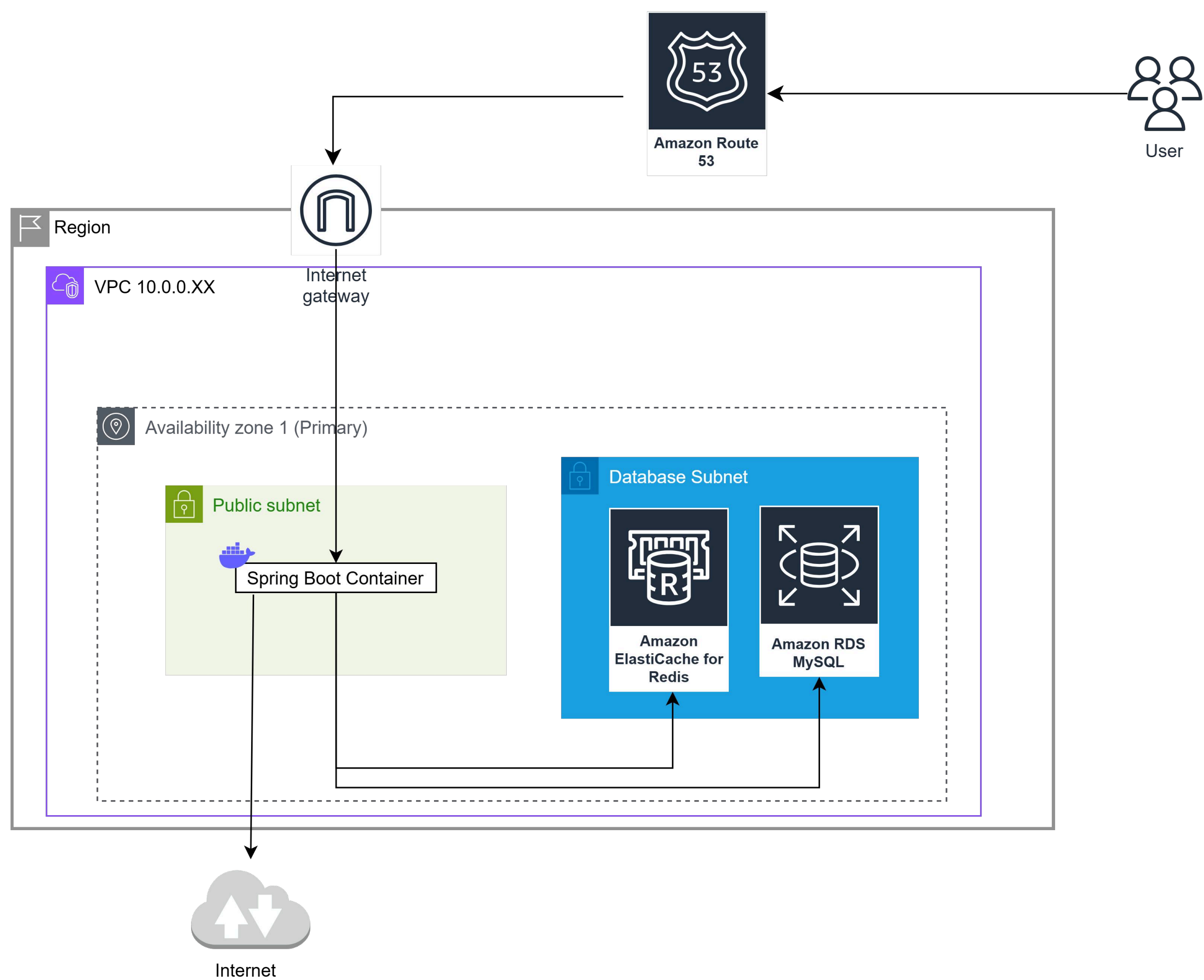


Docker Compose 기반 서비스 실행 순서 관리:



- **요구사항:**
Backend, MySQL, Redis, Data Loader 컨테이너가 안정적인 순서로 실행되어야 하며, DB 초기화 이전에 Backend가 실행되지 않아야 함.
- **작업:**
Docker Compose의 depends_on 및 healthcheck 기능을 활용하여 컨테이너 간 실행 순서를 관리
- **문제:**
MySQL 초기 실행이 완료되기 전에 Backend 또는 Data Loader가 먼저 실행될 경우 DB 연결 실패 및 초기 데이터 적재 오류가 발생할 수 있었음
- **문제 해결:**
 - a. MySQL 컨테이너에 healthcheck를 적용하여 DB 정상 상태 여부 확인
 - b. depends_on + service_healthy 조건을 통해 DB 준비 완료 이후 Data Loader 실행
 - c. Redis 실행 이후 Backend가 시작되도록 서비스 의존성 구성
 - d. 컨테이너 간 실행 순서를 명확히 분리하여 초기 실행 안정성 개선

AWS 초기 배포 아키텍처 (Single Server Architecture)



구조 특징

- Route53 기반 도메인 연결
- Spring Boot 서버를 Public Subnet에 직접 배포
- Redis 기반 Refresh Token 저장 및 캐시 구조 구성
- RDS를 별도 Database Subnet으로 분리하여 데이터 계층 보호
- 단일 서버 기반 구조로 초기 서비스 운영 비용 최소화

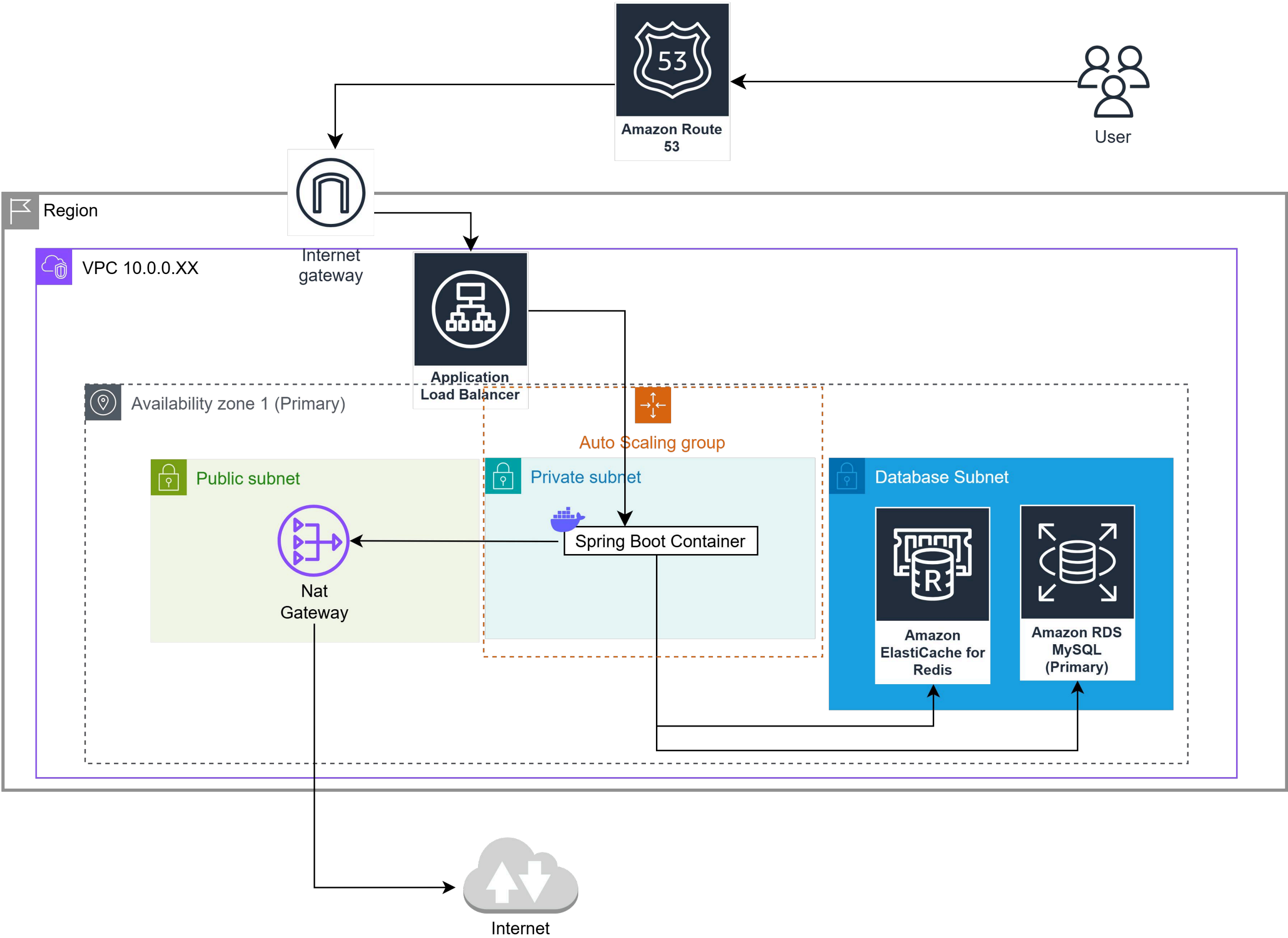
한계

- 단일 서버 장애 시 서비스 전체 중단 가능
- 트래픽 증가 시 수평 확장 어려움
- 서버가 Public Subnet에 직접 노출됨

추가 고려사항 - Redis 캐시 기반 병원 목록 조회 성능 개선 설계

병원 목록 조회는 사용자가 자주 접근하는 기능이지만, 병원 기본 정보는 자주 변경되지 않으므로 반복 조회 시 MySQL 부하를 줄일 필요가 있음
→ 조회 결과를 Redis에 캐싱한다면 DB부하 감소, API 응답 속도 개선, MySQL 병목 완화 등의 효과를 볼 수 있을 것으로 예상됨

AWS 확장형 아키텍처 (Load Balancing Architecture)



구조 특징

- Route53 + ALB 기반 트래픽 분산
- Spring Boot 서버를 Private Subnet 내부에 배치
- NAT Gateway를 통한 outbound 인터넷 통신 처리
- Redis 기반 Token Store 및 Cache 구조 적용
- JWT Stateless 인증 구조를 통해 수평 확장 대응
- Auto Scaling Group 기반 확장 가능 구조 설계

기대 효과

- 트래픽 증가 시 서버 수평 확장 가능
- 외부 직접 접근 제한으로 보안성 향상
- ALB 기반 부하 분산 처리 가능
- Stateless 인증 구조를 통한 서버 확장 대응 가능

Trade-off

- ALB, Auto Scaling, NAT Gateway, Private Subnet 등 구성 요소가 늘어나 설계와 운영 난이도 증가
- ALB, NAT Gateway, 다중 인스턴스 운영으로 초기 구조보다 비용이 높아짐
- 여러 서버가 동시에 운영되므로 배포 시 버전 불일치나 롤링 업데이트 전략을 고려해야 함
- Redis 캐시를 사용할 경우 데이터 변경 시 캐시 무효화 전략이 필요함.